



Universidad Autónoma de Nayarit

Unidad Académica De Economía

Sistemas Computacionales

“Semana 3: Listas Enlazadas Simples”

Jorge Armando Navarrete Ruiz

Docente.- DR. Eligardo Cruz Sanchez

U. A. ESTRUCTURA DE BASES DE DATOS

COMPRENDE.....	3
1 Visualización de Lista Enlazada.....	3
Checkpoint Metacognitivo - Comprende.....	3
APLICA.....	4
2 Implementación Guiada de Lista Enlazada.....	4
3 Resolver Problemas Clásicos con Depuración Visual.....	17
Checkpoint Metacognitivo - Aplica.....	23
4 Análisis de Casos de Uso Reales.....	23
Checkpoint Metacognitivo - Reflexiona.....	26
5 Validación de Implementación con Técnica de "Rubber Duck"	26
Checkpoint Metacognitivo - Valida.....	31
6 Diseño de Sistema: Lista con Acceso $O(1)$ al Final.....	31
Checkpoint Metacognitivo - Profundiza.....	31

COMPRENDE

1 Visualización de Lista Enlazada

Documento con los diagramas generados, anotados con tus propias palabras explicando:

(1) ¿Por qué insertar al inicio es $O(1)$?

1. Nos basamos en el insertar al inicio, una buena forma de explicarlo es en pseudocódigo o de forma gráfica, con esto podemos observar:
 - cómo se crea el nodo con valor
 - luego hacia donde apunta el valor del nodo nuevo
 - El valor delegado en nueva cabeza es el nuevo nodo
 - Y se actualiza el contador
2. Complejidad: $O(1)$ - siempre las mismas operaciones

(2) ¿Por qué insertar al final es $O(n)$?

1. Tomemos en cuenta el mismo diagrama, en este ejemplo hacemos una adición al ejecutable, en esencia puede ser la estructura if, en esto nos basamos de la siguiente sentencia.
función insertarFinal(valor):
| nuevoNodo $\leftarrow$ crearNodo(valor)
2. Con esto podemos empezar el análisis de la complejidad, en esta estructura nos basamos en el rellenado de la función:
 - Nos basamos en la lista vacía como un caso especial
 - En esta lista buscamos el último nodo
 - Y enlazamos al final el nuevo valor
3. Complejidad: $O(n)$ - debe recorrer toda la lista

(3) ¿Qué problema surge al eliminar el primer nodo que no ocurre con otros nodos?

1. Ocurre la necesidad de actualizar la referencia externa
2. No hay un nodo anterior: en este caso para eliminar un nodo hacemos `anterior.next  $\leftarrow$  actual.next`. En el caso del primer nodo no existe, por eso el código debe manejar este caso por separado
3. Resumen práctico: eliminar la cabeza requiere tratar explícitamente la variable de entrada y el caso lista-vacía; eliminar un nodo interior solo necesita re-enlazar el anterior y no tocar la raíz ni la posible variable

Checkpoint Metacognitivo - Comprende

¿Puedo explicar en mis propias palabras por qué una lista enlazada no permite acceso $O(1)$ por índice, aunque los arreglos sí?

El acceso por lista enlazada aloja de forma aleatoria los nodos de la información.

Mientras que si nos basamos en los arreglos podemos seguir en una secuencia ordenada ya que tenemos la ruta directa de donde se alojan esos datos.

Si me piden insertar un elemento entre el nodo 3 y el nodo 4 de una lista, ¿sé qué punteros debo modificar sin consultar código?

si, para este caso en específico lo que debemos hacer es enlazar el nuevo nodo hacia adelante con el puntero modificado y asignamos el valor nuevo.

¿Entiendo por qué necesito referencia al nodo ANTERIOR para eliminar un nodo en una lista simple?

Si, si no se hace así la eliminación del nodo puede verse implicada y puede tender a un fallo en el mismo ejecutable

APLICA

2 Implementación Guiada de Lista Enlazada

Código fuente completo de tu implementación con:

(1) Clase Nodo

(2) Clase ListaEnlazada con al menos 6 métodos funcionales

(3) Programa de prueba que demuestre cada operación

(4) Comentarios explicando la lógica de cada método

```
#include <iostream>
using namespace std;
```

```
/*
```

```
ESTRUCTURA NODO
```

Representa un elemento individual de la lista enlazada.

Atributos:

- dato: valor almacenado en el nodo (tipo int)
- siguiente: puntero al siguiente nodo en la cadena

Constructor:

- Inicializa el dato con el valor proporcionado
- Inicializa siguiente como nullptr

```
*/
```

```
struct Nodo {
    int dato;          // Valor almacenado
    Nodo* siguiente;   // Puntero al siguiente nodo

    // Constructor: crea un nodo con un valor inicial
    Nodo(int valor) {
```

```

        dato = valor;
        siguiente = nullptr; // Sin enlace inicial
    }
};

/*
CLASE LISTAENLAZADA
Administra una colección dinámica de nodos enlazados.

Atributos privados:
- cabeza: puntero al primer nodo (nullptr si está vacía)
- tamaño: contador de elementos en la lista

Métodos públicos:
1. Constructor/Destructor: gestión de memoria
2. obtenerTamano(): retorna el número de elementos
3. estaVacía(): verifica si la lista está vacía
4. imprimir(): muestra todos los elementos
5. insertarInicio(valor): agrega elemento al inicio - O(1)
6. insertarFinal(valor): agrega elemento al final - O(n)
7. eliminarInicio(): elimina el primer elemento - O(1)
8. buscar(valor): busca un valor en la lista - O(n)
*/

class ListaEnlazada {
private:
    Nodo* cabeza; // Puntero al primer nodo
    int tamaño; // Número de elementos en la lista

public:
    /*
        MÉTODO 1: CONSTRUCTOR
        Inicializa una lista enlazada vacía.

        Lógica:
        - cabeza = nullptr (no hay primer nodo)
        - tamaño = 0 (lista vacía)

        Complejidad: O(1)
    */
    ListaEnlazada() {
        cabeza = nullptr;
        tamaño = 0;
    }

```

```
}
```

```
/*
```

MÉTODO 2: DESTRUCTOR

Libera toda la memoria ocupada por los nodos al destruir la lista.
Se ejecuta automáticamente cuando el objeto sale de scope.

Lógica:

1. Empezar en cabeza
2. Para cada nodo:
 - Guardar referencia al siguiente
 - Eliminar el nodo actual con delete
 - Avanzar al siguiente
3. Continuar hasta llegar a nullptr

Importancia: Evita memory leaks (fugas de memoria)

Complejidad: $O(n)$ - debe liberar todos los nodos

```
*/
```

```
~ListaEnlazada() {
```

```
    Nodo* actual = cabeza;
```

```
    while (actual != nullptr) {
```

```
        Nodo* siguiente = actual->siguiente; // Guardar siguiente antes de delete
```

```
        delete actual; // Liberar memoria del nodo actual
```

```
        actual = siguiente; // Avanzar al siguiente
```

```
    }
```

```
    // Opcional: establecer cabeza y tamaño (aunque el objeto se destruye)
```

```
    cabeza = nullptr;
```

```
    tamaño = 0;
```

```
}
```

```
/*
```

MÉTODO 3: OBTENER TAMAÑO

Retorna el número de elementos en la lista.

Lógica:

- Simplemente retorna el atributo tamaño (mantenido actualizado)

Ventaja: $O(1)$ gracias al contador

Sin contador tendríamos que recorrer la lista completa - $O(n)$

Complejidad: $O(1)$

```
*/  
int obtenerTamano() {  
    return tamano;  
}
```

```
/*  
    MÉTODO 4: ESTÁ VACÍA
```

Verifica si la lista no tiene elementos.

Lógica:

- Retorna true si cabeza == nullptr
- Retorna false si cabeza apunta a algún nodo

Alternativa equivalente: `return tamano == 0;`

Uso: Verificar antes de operaciones como eliminar o acceder

Complejidad: $O(1)$

```
*/  
  
bool estaVacia() {  
    return cabeza == nullptr;  
}
```

```
/*  
    MÉTODO 5: IMPRIMIR
```

Muestra todos los elementos de la lista en orden.

Formato: Lista: 10 -> 20 -> 30 -> null

Lógica:

1. Verificar si está vacía (caso especial)
2. Recorrer desde cabeza hasta nullptr
3. Para cada nodo:
 - Imprimir su dato
 - Imprimir flecha si no es el último
 - Avanzar al siguiente
4. Imprimir "null" al final

Complejidad: $O(n)$ - debe recorrer todos los nodo
*/

```
void imprimir() {  
    // Caso especial: lista vacía  
    if (estaVacía()) {  
        cout << "Lista vacía" << endl;  
        return;  
    }  
  
    // Recorrer e imprimir  
    Nodo* actual = cabeza;  
    cout << "Lista: ";  
  
    while (actual != nullptr) {  
        cout << actual->dato;        // Imprimir dato  
  
        if (actual->siguiente != nullptr) {  
            cout << " -> ";        // Flecha entre nodos  
        }  
  
        actual = actual->siguiente;    // Avanzar  
    }  
  
    cout << " -> null" << endl;    // Indicar final  
}  
  
/*
```

MÉTODO 6: INSERTAR AL INICIO

Agrega un nuevo elemento al inicio de la lista.

Lógica:

1. Crear un nuevo nodo con el valor dado
2. Hacer que nuevo->siguiente apunte a cabeza
(si la lista estaba vacía, cabeza era nullptr → correcto)
3. Actualizar cabeza para que apunte al nuevo nodo
4. Incrementar tamaño

Visualización:

ANTES: cabeza -> [A] -> [B] -> null

CREAR: nuevo -> [X] -> null
PASO 2: nuevo -> [X] -> [A] -> [B] -> null
PASO 3: cabeza -> [X] -> [A] -> [B] -> null

Ventaja: No requiere recorrido, solo cambiar punteros
Complejidad: $O(1)$ - tiempo constante

```
*/  
void insertarInicio(int valor) {  
    // Paso 1: Crear nuevo nodo  
    Nodo* nuevoNodo = new Nodo(valor);  
  
    // Paso 2: Enlazar nuevo nodo al antiguo primero  
    nuevoNodo->siguiente = cabeza;  
  
    // Paso 3: Actualizar cabeza  
    cabeza = nuevoNodo;  
  
    // Paso 4: Incrementar contador  
    tamano++;  
}
```

```
/*  
MÉTODO 7: INSERTAR AL FINAL  
Agrega un nuevo elemento al final de la lista.
```

Lógica:

1. Crear nuevo nodo
2. Caso especial: si la lista está vacía
 - Hacer cabeza = nuevoNodo
3. Caso general: lista con elementos
 - Recorrer hasta encontrar el último nodo
 - Hacer ultimo->siguiente = nuevoNodo
4. Incrementar tamaño

Visualización del recorrido:

cabeza -> [A] -> [B] -> [C] -> null
 ↓ ↓ ↓ (último, siguiente == nullptr)
 paso1 paso2 PARAR → enlazar aquí

Desventaja: Requiere recorrer toda la lista
Mejora posible: mantener puntero "cola" → $O(1)$
Complejidad: $O(n)$ - debe recorrer hasta el final

*/

```
void insertarFinal(int valor) {  
    // Paso 1: Crear nuevo nodo  
    Nodo* nuevoNodo = new Nodo(valor);  
  
    // Caso especial: lista vacía  
    if (estaVacia()) {  
        cabeza = nuevoNodo;  
        tamano++;  
        return;  
    }  
  
    // Caso general: encontrar el último nodo  
    Nodo* actual = cabeza;  
  
    // Recorrer hasta el último (siguiente == nullptr)  
    while (actual->siguiente != nullptr) {  
        actual = actual->siguiente;  
    }  
  
    // Enlazar el nuevo nodo al final  
    actual->siguiente = nuevoNodo;  
  
    // Incrementar contador  
    tamano++;  
}  
  
/*
```

MÉTODO 8: ELIMINAR AL INICIO

Elimina el primer elemento de la lista.
Retorna true si fue exitoso, false si la lista estaba vacía.

Lógica:

1. Verificar si está vacía (error si lo está)
2. Guardar referencia al nodo a eliminar
3. Actualizar cabeza al segundo nodo
4. Liberar memoria del nodo eliminado con delete
5. Decrementar tamano

Visualización:

ANTES: cabeza -> [A] -> [B] -> [C] -> null

GUARDAR: temp -> [A]
PASO 3: cabeza -> [B] -> [C] -> null
PASO 4: delete temp (liberar [A])

IMPORTANTE: El orden es crucial

- Primero guardar referencia
- Luego actualizar cabeza
- Finalmente hacer delete

Si hacemos delete antes de actualizar cabeza → perdemos la lista

Complejidad: $O(1)$ - no requiere recorrido

```
*/  
bool eliminarInicio() {  
    // Verificación: lista vacía  
    if (estaVacia()) {  
        cout << "Error: No se puede eliminar de una lista vacía" << endl;  
        return false;  
    }  
  
    // Paso 1: Guardar referencia al nodo a eliminar  
    Nodo* nodoAEliminar = cabeza;  
  
    // Paso 2: Actualizar cabeza  
    cabeza = cabeza->siguiente;  
  
    // Paso 3: Liberar memoria (CRÍTICO para evitar memory leak)  
    delete nodoAEliminar;  
  
    // Paso 4: Decrementar contador  
    tamano--;  
  
    return true; // Operación exitosa  
}
```

```
/*  
MÉTODO 9: BUSCAR
```

Busca un valor en la lista.
Retorna true si existe, false si no existe.

Lógica:

1. Si la lista está vacía, retornar false

2. Recorrer desde cabeza hasta nullptr
3. En cada nodo:
 - Comparar dato con valor buscado
 - Si coincide, retornar true inmediatamente
 - Si no coincide, avanzar al siguiente
4. Si llegamos al final sin encontrar, retornar false

Visualización (buscar 20):

cabeza -> [10] -> [20] -> [30] -> null

↓ ↓
 ¿10=20? ¿20=20? → ¡SÍ! return true
 No

Optimización: Retorno temprano

- No seguimos buscando después de encontrar

Casos de complejidad:

- Mejor caso: $O(1)$ - elemento en primera posición
- Peor caso: $O(n)$ - elemento al final o no existe
- Promedio: $O(n/2) \rightarrow O(n)$

Complejidad: $O(n)$ - búsqueda lineal

*/

```

bool buscar(int valor) {
    // Caso especial: lista vacía
    if (estaVacia()) {
        return false;
    }

    // Recorrer la lista
    Nodo* actual = cabeza;

    while (actual != nullptr) {
        // Comparar con el valor buscado
        if (actual->dato == valor) {
            return true; // ¡Encontrado! Retorno temprano
        }

        // Avanzar al siguiente
        actual = actual->siguiente;
    }
  
```

```

        // Si salimos del bucle, no se encontró
        return false;
    }
};

/*
PROGRAMA DE PRUEBA

Demuestra todas las operaciones de la lista enlazada con casos variados.

Pruebas incluidas:
1. Estado inicial (lista vacía)
2. Inserción al inicio
3. Inserción al final
4. Búsqueda de elementos existentes y no existentes
5. Eliminación al inicio
6. Búsqueda después de eliminar
7. Vaciar la lista completamente
8. Operaciones en lista vacía (manejo de errores)

*/
int main() {
    // Crear lista enlazada
    ListaEnlazada lista;

    cout << "===== " << endl;
    cout << " PRUEBA COMPLETA DE LISTA ENLAZADA " << endl;
    cout << "===== " << endl <<
endl;

    // PRUEBA 1: Estado inicial (lista vacía)
    //
    cout << " " << endl;
    cout << " PRUEBA 1: Estado inicial | " << endl;
    cout << " " << endl;
    lista.imprimir();
    cout << "Tamaño: " << lista.obtenerTamano() << endl;
    cout << "¿Está vacía? " << (lista.estaVacia() ? "Sí" : "No") << endl;
    cout << endl;

    // PRUEBA 2: Insertar al INICIO
    cout << " " << endl;

```

```

cout << " | PRUEBA 2: Insertar al INICIO | " << endl;
cout << " | " << endl;
cout << "Insertando: 30, 20, 10" << endl;
lista.insertarInicio(30);
lista.insertarInicio(20);
lista.insertarInicio(10);
lista.imprimir();
cout << "Tamaño: " << lista.obtenerTamano() << endl;
cout << "Explicación: Los elementos se insertan al inicio," << endl;
cout << "por lo que el orden final es INVERSO al de inserción." << endl;
cout << endl;

```

// PRUEBA 3: Insertar al FINAL

```

cout << " | " << endl;
cout << " | PRUEBA 3: Insertar al FINAL | " << endl;
cout << " | " << endl;
cout << "Insertando: 40, 50" << endl;
lista.insertarFinal(40);
lista.insertarFinal(50);
lista.imprimir();
cout << "Tamaño: " << lista.obtenerTamano() << endl;
cout << "Explicación: insertarFinal mantiene el orden de inserción" << endl;
cout << "pero es O(n) porque debe recorrer hasta el final." << endl;
cout << endl;

```

// PRUEBA 4: Búsqueda de elementos

```

cout << " | " << endl;
cout << " | PRUEBA 4: Búsqueda de elementos | " << endl;
cout << " | " << endl;

```

```

cout << "Elementos existentes:" << endl;
int existentes[] = {10, 20, 30, 40, 50};
for (int i = 0; i < 5; i++) {
    bool encontrado = lista.buscar(existentes[i]);
    cout << " • Buscar " << existentes[i] << ": "
        << (encontrado ? "✓ ENCONTRADO" : "✗ NO encontrado") << endl;
}

```

```

cout << "\nElementos NO existentes:" << endl;
int noExistentes[] = {5, 15, 99};

```

```

for (int i = 0; i < 3; i++) {
    bool encontrado = lista.buscar(noExistentes[i]);
    cout << " • Buscar " << noExistentes[i] << ": "
        << (encontrado ? "✓ ENCONTRADO" : "✗ NO encontrado") << endl;
}
cout << endl;

```

// PRUEBA 5: Eliminar al inicio (múltiples veces)

```

cout << " " << endl;
cout << " PRUEBA 5: Eliminar al INICIO | " << endl;
cout << " " << endl;

```

```

cout << "Eliminando el primer elemento:" << endl;
bool resultado = lista.eliminarInicio();
cout << "Resultado: " << (resultado ? "✓ Éxito" : "✗ Falló") << endl;
lista.imprimir();
cout << "Tamaño: " << lista.obtenerTamano() << endl;

```

```

cout << "\nEliminando otro elemento:" << endl;
resultado = lista.eliminarInicio();
cout << "Resultado: " << (resultado ? "✓ Éxito" : "✗ Falló") << endl;
lista.imprimir();
cout << "Tamaño: " << lista.obtenerTamano() << endl;
cout << endl;

```

// PRUEBA 6: Búsqueda después de eliminar

```

cout << " " << endl;
cout << " PRUEBA 6: Búsqueda tras eliminar | " << endl;
cout << " " << endl;
cout << "Buscar 10 (fue eliminado): "
    << (lista.buscar(10) ? "✓ Encontrado" : "✗ NO encontrado") << endl;
cout << "Buscar 20 (fue eliminado): "
    << (lista.buscar(20) ? "✓ Encontrado" : "✗ NO encontrado") << endl;
cout << "Buscar 30 (aún existe): "
    << (lista.buscar(30) ? "✓ Encontrado" : "✗ NO encontrado") << endl;
cout << endl;

```

// PRUEBA 7: Vaciar completamente la lista

```

cout << " " << endl;
cout << " PRUEBA 7: Vaciar la lista | " << endl;

```

```
cout << " | " << endl;
cout << "Eliminando todos los elementos restantes..." << endl;
```

```
int contador = 0;
while (!lista.estaVacia()) {
    lista.eliminarInicio();
    contador++;
}
```

```
cout << "Elementos eliminados: " << contador << endl;
lista.imprimir();
cout << "Tamaño: " << lista.obtenerTamano() << endl;
cout << "¿Está vacía? " << (lista.estaVacia() ? "Sí" : "No") << endl;
cout << endl;
```

```
// PRUEBA 8: Operaciones en lista vacía (manejo de errores)
```

```
cout << " | " << endl;
cout << " | PRUEBA 8: Operaciones en lista vacía | " << endl;
cout << " | " << endl;
```

```
cout << "Intentar eliminar de lista vacía:" << endl;
resultado = lista.eliminarInicio();
cout << "Resultado: " << (resultado ? "✓ Éxito" : "✗ Falló (esperado)") << endl;
```

```
cout << "\nBuscar en lista vacía:" << endl;
cout << "Buscar 10: " << (lista.buscar(10) ? "Encontrado" : "NO encontrado
(esperado)") << endl;
cout << endl;
// PRUEBA 9: Reconstruir la lista
```

```
cout << " | " << endl;
cout << " | PRUEBA 9: Reconstruir la lista | " << endl;
cout << " | " << endl;
cout << "Insertando 5 elementos al final: 100, 200, 300, 400, 500" << endl;
lista.insertarFinal(100);
lista.insertarFinal(200);
lista.insertarFinal(300);
lista.insertarFinal(400);
lista.insertarFinal(500);
lista.imprimir();
cout << "Tamaño final: " << lista.obtenerTamano() << endl;
cout << endl;
```



```
    return 0;
}
```

3 Resolver Problemas Clásicos con Depuración Visual

Implementación de al menos 2 problemas clásicos (invertir lista + encontrar medio) con:

(1) Código funcional probado

```
#include <iostream>
```

```
using namespace std;
```

```
struct Nodo {
    int dato;
    Nodo* siguiente;

    Nodo(int valor) {
        dato = valor;
        siguiente = nullptr;
    }
};
```

```
class ListaEnlazada {
private:
    Nodo* cabeza;
    int tamano;
```

```
public:
    ListaEnlazada() {
        cabeza = nullptr;
        tamano = 0;
    }

    ~ListaEnlazada() {
        Nodo* actual = cabeza;
        while (actual != nullptr) {
            Nodo* siguiente = actual->siguiente;
            delete actual;
            actual = siguiente;
        }
    }
```

```
    bool estaVacia() {
        return cabeza == nullptr;
    }
```

```
    int obtenerTamano() {
        return tamano;
    }
```

```

void insertarFinal(int valor) {
    Nodo* nuevoNodo = new Nodo(valor);

    if (cabeza == nullptr) {
        cabeza = nuevoNodo;
        tamano++;
        return;
    }

    Nodo* actual = cabeza;
    while (actual->siguiente != nullptr) {
        actual = actual->siguiente;
    }
    actual->siguiente = nuevoNodo;
    tamano++;
}

void imprimir() {
    if (cabeza == nullptr) {
        cout << "Lista vacía" << endl;
        return;
    }

    Nodo* actual = cabeza;
    cout << "cabeza → ";
    while (actual != nullptr) {
        cout << "[" << actual->dato << "]";
        if (actual->siguiente != nullptr) {
            cout << " → ";
        }
        actual = actual->siguiente;
    }
    cout << " → null" << endl;
}

// ===== PROBLEMA 1: INVERTIR LISTA =====
void invertir() {
    Nodo* anterior = nullptr;
    Nodo* actual = cabeza;
    Nodo* siguiente = nullptr;

    while (actual != nullptr) {
        siguiente = actual->siguiente; // Guardar siguiente
        actual->siguiente = anterior; // Invertir enlace
        anterior = actual;           // Avanzar anterior
        actual = siguiente;          // Avanzar actual
    }
}

```

```

        cabeza = anterior; // Actualizar cabeza
    }

// ===== PROBLEMA 2: ENCONTRAR MEDIO =====
Nodo* encontrarMedio() {
    if (cabeza == nullptr) {
        return nullptr;
    }

    Nodo* lento = cabeza;
    Nodo* rapido = cabeza;

    // Técnica tortuga y liebre
    while (rapido != nullptr && rapido->siguiente != nullptr) {
        lento = lento->siguiente; // Avanza 1
        rapido = rapido->siguiente->siguiente; // Avanza 2
    }

    return lento; // Nodo del medio
}

void mostrarMedio() {
    Nodo* medio = encontrarMedio();

    if (medio == nullptr) {
        cout << "Lista vacía, no hay nodo del medio" << endl;
    } else {
        cout << "Nodo del medio: [" << medio->dato << "]" << endl;
    }
}

};

int main() {
    cout <<
    "=====
<< endl;
    cout << "    PROBLEMAS CLÁSICOS: LISTAS ENLAZADAS" << endl;
    cout <<
    "=====
    " << endl;

    // ===== PRUEBAS: INVERTIR LISTA =====
    cout << "
    cout << "    PROBLEMA 1: INVERTIR LISTA    " << endl;
    cout << "
    ListaEnlazada lista1;

```

```

cout << "Prueba 1: Lista vacía" << endl;
lista1.imprimir();
lista1.invertir();
cout << "Después de invertir:" << endl;
lista1.imprimir();
cout << endl;

```

```

cout << "Prueba 2: Un elemento" << endl;
lista1.insertarFinal(10);
lista1.imprimir();
lista1.invertir();
cout << "Después de invertir:" << endl;
lista1.imprimir();
cout << endl;

```

```

cout << "Prueba 3: Múltiples elementos" << endl;
ListaEnlazada lista2;
lista2.insertarFinal(1);
lista2.insertarFinal(2);
lista2.insertarFinal(3);
lista2.insertarFinal(4);
lista2.insertarFinal(5);
cout << "ANTES:" << endl;
lista2.imprimir();
lista2.invertir();
cout << "DESPUÉS:" << endl;
lista2.imprimir();
cout << "\nVerificación: invertir nuevamente" << endl;
lista2.invertir();
lista2.imprimir();
cout << endl;

```

```

// ===== PRUEBAS: ENCONTRAR MEDIO =====

```

```

cout << " " << endl;
cout << " || PROBLEMA 2: ENCONTRAR MEDIO || " << endl;
cout << " || \n" << endl;

```

```

ListaEnlazada lista3;

```

```

cout << "Prueba 1: Lista vacía" << endl;
lista3.imprimir();
lista3.mostrarMedio();
cout << endl;

```

```

cout << "Prueba 2: Un elemento" << endl;
lista3.insertarFinal(100);
lista3.imprimir();

```

```

lista3.mostrarMedio();
cout << endl;

cout << "Prueba 3: Dos elementos (par)" << endl;
lista3.insertarFinal(200);
lista3.imprimir();
lista3.mostrarMedio();
cout << endl;

cout << "Prueba 4: Tres elementos (impar)" << endl;
lista3.insertarFinal(300);
lista3.imprimir();
lista3.mostrarMedio();
cout << endl;

cout << "Prueba 5: Cinco elementos (impar)" << endl;
ListaEnlazada lista4;
lista4.insertarFinal(10);
lista4.insertarFinal(20);
lista4.insertarFinal(30);
lista4.insertarFinal(40);
lista4.insertarFinal(50);
lista4.imprimir();
lista4.mostrarMedio();
cout << "Esperado: [30]" << endl << endl;

cout << "Prueba 6: Seis elementos (par)" << endl;
lista4.insertarFinal(60);
lista4.imprimir();
lista4.mostrarMedio();
cout << "Esperado: [40]" << endl << endl;

// ===== PRUEBA COMBINADA =====
cout << " " << endl;
cout << " || PRUEBA COMBINADA: AMBOS MÉTODOS || " << endl;
cout << " || \n" << endl;

ListaEnlazada lista5;
lista5.insertarFinal(5);
lista5.insertarFinal(10);
lista5.insertarFinal(15);
lista5.insertarFinal(20);
lista5.insertarFinal(25);

cout << "Lista original:" << endl;
lista5.imprimir();
cout << "Tamaño: " << lista5.obtenerTamano() << endl;
lista5.mostrarMedio();

```

```

    cout << "\nDespués de invertir:" << endl;
    lista5.invertir();
    lista5.imprimir();
    lista5.mostrarMedio();

    cout <<
    "\n=====
" << endl;
    cout << "          FIN DE LAS PRUEBAS" << endl;
    cout <<
    "=====
" << endl;

    return 0;
}

```

(2) Documento explicando el algoritmo paso a paso en tus propias palabras

1. Cuando recién se ejecuta el struct nodo, lo que ocurre es que dato guarda un valor, siguiente actúa de puntero al siguiente nodo, y constructor inicializa y pone siguiente en null, cada uno va distribuyendo sus valores en reasignación.
2. Al inicializar lista enlazada se crea una lista vacía, luego de esto se abre otra lista enlazada que recorre todos los nodos y los libera de memoria uno por uno.
3. Luego pasamos al estaVacía bool, esto verifica que cabeza sea un valor null.
4. Luego está void insertarFinal, esto crea un nuevo nodo, si la lista está vacía el nuevo nodo es la cabeza, sino recorre hasta el final.
5. Void imprimir recorre la lista e imprime cada nodo con flechas
6. void mostrarMedio llama a encontrarMedio(), si retorna null imprime el mensaje de esta vacío, sino imprime su valor

(3) Análisis de complejidad temporal y espacial.

Para invertir lista es:

Complejidad Temporal: $O(n)$

Complejidad espacial: $O(1)$

Para encontrar medio es:

Complejidad Temporal: $O(n)$

Complejidad espacial: $O(1)$

Checkpoint Metacognitivo - Aplica

¿Puedo implementar insertarInicio() sin mirar el código? ¿Qué línea escribiría primero y por qué en ese orden?

Si tuviera un mejor dominio sobre las reglas de los punteros si podría, pero aun asi podria si me baso en que solo se debe tocar la cabeza, una idea más clara es decir que el nuevo nodo debe apuntar al viejo primer nodo, luego cabeza debe apuntar al nuevo nodo.

Nodo* nuevo = new Nodo(valor); esta sería la primera línea, porque necesitas tener el nodo creado antes de poder enlazarlo con algo

Si mi método eliminar() no funciona para el primer elemento, ¿puede identificar qué caso especial olvidé?

No, siendo sincero y analizando el código no logro encontrar una forma de agregar un caso especial.

¿Cuánto del código escribí yo versus cuánto copié directamente de la IA sin modificar?

en su mayoría fueron ediciones de la ia, tenia problemas en la estructuracion e implementacion de las listas enlazadas y encontrar medio

4 Análisis de Casos de Uso Reales

Documento comparativo que incluya:

(1) Tu análisis inicial de cada escenario ANTES de consultar la IA

Escenario A: Playlist de música

¿Qué estructura elegirías y por qué?

Lista doblemente enlazada, porque una playlist tiene que avanzar a la siguiente, regresar, insertar, y eliminar canción.

Escenario B: Buffer de teclado

¿Qué estructura elegirías y por qué?

Cola, porque escribes en el teclado la primera tecla presionada es la primera en procesarse, y se sigue en el orden de llegada

Escenario C: Sistema de versiones de documento

¿Qué estructura elegirías y por qué?

Pila, porque es un sistema de versiones, la ultima acción realizada es la primera en deshacerse.

(2) El análisis de la IA

A) Sistema de playlist de música

Operaciones principales:

Insertar canciones al final (alta)

Eliminar la canción actual (media)

Navegar siguiente/anterior (alta)

Acceder a la canción actual (alta)

Frecuencia relativa:

Inserciones: Alta (usuarios agregan canciones constantemente)

Eliminaciones: Media (ocasional, pero importante)

Navegación: Alta (usuarios suelen moverse entre canciones)

Requisitos de memoria:

Tamaño pequeño/mediano (50-500 elementos)

Punteros para cada nodo (en listas enlazadas), arrays más compactos

Recomendación:

Lista Doblemente Enlazada

Justificación:

Insertar/eliminar la canción actual en $O(1)$ si se tiene referencia al nodo.

Navegación eficiente hacia adelante y atrás.

Arrays requieren movimientos costosos al eliminar/inserciones en medio.

Doble enlace permite fácil acceso bidireccional.

B) Buffer de teclado

Operaciones principales:

Insertar al final (alta, cada caracter)

Procesar (eliminar) al inicio (alta)

Consulta del tamaño (media)

Acceso secuencial en orden FIFO

Frecuencia relativa:

Insertar: Alta

Eliminar: Alta

Tamaño: Media

Requisitos de memoria:

Muy pequeño: Máximo 256 caracteres.

Recomendación:

Arreglo Circular (Circular Buffer)

Justificación:

$O(1)$ inserción y eliminación.

Estructura optimizada y muy compacta para buffers.

Latencia mínima: no hay punteros, memoria cache-friendly.

C) Sistema de versiones de documento

Operaciones principales:

Insertar nueva versión al final (alta)
Acceder a la última versión (alta)
Navegar hacia versiones anteriores (media)
Nunca se elimina del medio

Frecuencia relativa:

Insertar: Alta
Acceder último: Alta
Navegar: Media

Requisitos de memoria:

Cada nodo debe almacenar un documento grande.
Punteros extra son irrelevantes respecto al tamaño del documento.

Recomendación:

Lista Enlazada Simple

Justificación:

Insertar al final es $O(1)$ (con puntero a tail).
Acceso a último $O(1)$.
Navegar hacia atrás requiere paso secuencial, pero aceptable si es poco frecuente.
No se requieren eliminaciones en medio ni movimientos complejos.

(3) Tu reflexión sobre las diferencias entre ambos análisis**Escenario A (Playlist): SÍ - Coincidencia total**

Ambos elegimos: Lista doblemente enlazada

Escenario B (Buffer): COINCIDENCIA CONCEPTUAL, diferencia en implementación

Elección: Cola (Queue) - concepto abstracto

IA elección: Arreglo Circular - implementación concreta

Escenario C (Versiones): NO - Desacuerdo significativo

Elección: Pila (Stack) - LIFO

IA elección: Lista Enlazada Simple - estructura flexible

(4) Conclusiones sobre cuándo usar cada estructura.

- Lista Doblemente Enlazada: Ideal para escenas con navegación bidireccional y eliminaciones/inserciones en medio ([playlist]).
- Arreglo Circular: Para buffers pequeños y críticos en latencia ([buffer teclado]).
- Lista Enlazada Simple: Para historiales/versiones donde solo agregas al final y navegas ocasionalmente ([versiones de documento]).

Checkpoint Metacognitivo - Reflexiona

Si alguien me pregunta "¿es mejor usar arreglo o lista enlazada?", ¿puedo responder "depende" y explicar de qué depende?

Si, aunque aun me parezcan confusas las ideas clave de cada uno aun puedo dar una opinión cercana de lo son cada una.

¿Puedo identificar al menos 2 escenarios donde elegir la estructura incorrecta causaría problemas graves de rendimiento?

Si, en dado caso de un sistema de comercio y un editor de texto, estos al ser acomodados con una estructura incorrecta pueden llevar a fallas catastróficas.

Cuando la IA me dio una recomendación, ¿la acepté automáticamente o cuestioné su razonamiento? la acepte inmediatamente

5 Validación de Implementación con Técnica de "Rubber Duck"

Transcripción de la sesión de "rubber duck debugging" con al menos 3 métodos diferentes. Para cada método, incluye:

(1) El código original

```
#include <iostream>
using namespace std;
```

```
struct Nodo {
    int dato;
    Nodo* siguiente;

    Nodo(int valor) {
        dato = valor;
        siguiente = nullptr;
    }
};
```

```
class ListaEnlazada {
private:
    Nodo* cabeza;
    int tamano;

public:
    ListaEnlazada() {
        cabeza = nullptr;
```

```

    tamano = 0;
}

void insertarFinal(int valor) {
    Nodo* nuevoNodo = new Nodo(valor);

    if (cabeza == nullptr) {
        cabeza = nuevoNodo;
        tamano++;
        return;
    }

    Nodo* actual = cabeza;
    while (actual->siguiente != nullptr) {
        actual = actual->siguiente;
    }
    actual->siguiente = nuevoNodo;
    tamano++;
}
};

```

(2) Las preguntas de la IA y tus respuestas

Pregunta 1: ¿Qué hace la primera línea?

Nodo* nuevoNodo = new Nodo(valor);

Respuesta del estudiante:

"Necesitamos tener listo el nodo antes de conectarlo a la lista. La memoria debe reservarse dinámicamente (new) para que el nodo exista después de salir de la función."

✅ Evaluación: Correcto. Memoria dinámica en heap persiste después de retornar.

Pregunta 2: ¿Qué significa cabeza == nullptr?

Respuesta del estudiante:

"Significa que la lista está vacía. cabeza es un puntero al primer nodo."

✅ Evaluación: Correcto.

Pregunta 3: ¿Por qué la condición es actual->siguiente != nullptr y no actual != nullptr?

Respuesta del estudiante:

"Porque necesitamos quedarnos en el último nodo, no pasar más allá. Si usas `actual != nullptr`, terminarías con `actual = nullptr` (error). Con `actual->siguiente != nullptr`, te detienes exactamente en el último nodo."

✓ Evaluación: Perfecto. Evita dereferencia de `nullptr`.

Pregunta 4: ¿A qué nodo apunta actual al salir del while?

Ejemplo: `[1] → [2] → [3] → nullptr`

Respuesta del estudiante:

"Apunta al nodo 2, que es el último. El ciclo se detiene justo cuando encuentra que el siguiente es `nullptr`."

✓ Evaluación: Correcto. `actual` apunta al último nodo válido.

Pregunta 5: ¿Cuál es la complejidad de insertar N elementos?

Respuesta del estudiante:

"Es $O(N^2)$. Porque cada vez que insertas, recorres toda la lista hasta el final."

✓ Evaluación: Correcto.

Pregunta 6: ¿Cómo optimizar a $O(1)$?

Respuesta del estudiante:

"Necesitamos un puntero al último nodo, así no recorremos toda la lista."

✓ Evaluación: ¡Exacto! Puntero cola resuelve el problema.

Pregunta 7: ¿Necesitas un destructor?

Respuesta del estudiante:

"Sí. Porque usas `new` y debes liberar memoria con `delete`."

✓ Evaluación: Correcto. Previene memory leaks.

(3) Bugs o mejoras que descubriste durante la explicación

Mejora #1: Puntero cola para $O(1)$

Solución:

C++

```
class ListaEnlazada {
```

```
private:
```

```
    Nodo* cabeza;
```

```
    Nodo* cola;    // Agregar puntero al último nodo
```

```
    int tamano;
```

```
};
```

Mejora #2: Destructor correcto

Solución:

C++

```
~ListaEnlazada() {
```

```

Nodo* actual = cabeza;
while (actual != nullptr) {
    Nodo* siguiente = actual->siguiente; // Guardar referencia
    delete actual; // Liberar
    actual = siguiente; // Avanzar
}
}

```

(4) Código corregido si aplica.

```

#include <iostream>
using namespace std;

```

```

struct Nodo {
    int dato;
    Nodo* siguiente;

    Nodo(int valor) {
        dato = valor;
        siguiente = nullptr;
    }
};

```

```

class ListaEnlazada {
private:
    Nodo* cabeza;
    Nodo* cola; // Optimización: puntero al último nodo
    int tamano;

```

```

public:
    // Constructor
    ListaEnlazada() {
        cabeza = nullptr;
        cola = nullptr; // Inicializar cola
        tamano = 0;
    }

```

// OPTIMIZADO: O(1) por inserción

```

void insertarFinal(int valor) {
    Nodo* nuevoNodo = new Nodo(valor);

```

// Caso 1: lista vacía

```

if (cabeza == nullptr) {
    cabeza = nuevoNodo;
    cola = nuevoNodo; // Ambos apuntan al único nodo

```

```

    }
    else {
        // Caso 2: lista con elementos
        cola->siguiente = nuevoNodo; // O(1) - sin recorrer
        cola = nuevoNodo;          // Actualizar cola
    }

    tamano++;
}

// DESTRUCTOR: libera toda la memoria
~ListaEnlazada() {
    Nodo* actual = cabeza;
    while (actual != nullptr) {
        Nodo* siguiente = actual->siguiente;
        delete actual;
        actual = siguiente;
    }
}

// Método auxiliar para debugging
void imprimir() const {
    Nodo* actual = cabeza;
    cout << "Lista: ";
    while (actual != nullptr) {
        cout << actual->dato << " -> ";
        actual = actual->siguiente;
    }
    cout << "nullptr" << endl;
}

};

int main() {
    ListaEnlazada lista;

    lista.insertarFinal(1);
    lista.insertarFinal(2);
    lista.insertarFinal(3);
    lista.imprimir(); // Lista: 1 -> 2 -> 3 -> nullptr

    return 0; // Destructor se llama automáticamente
}

```

Checkpoint Metacognitivo - Valida

¿Mi implementación pasa TODOS los casos de prueba de la secuencia anterior sin errores?

No, aun hay estructuras en donde fallo como tambien en la implementación

Cuando expliqué mi código a la IA, ¿hubo partes que no pude explicar claramente? ¿Por qué?

hubo una parte donde analizamos porque guardar siguiente antes de hacer delete en el destructor, no tengo aun claro porque hacerlo

¿Escribí pruebas yo mismo o solo usé las que me dieron? ¿Puedo pensar en un caso de prueba adicional?

No escribí pruebas ejecutables, solo respondí teoría y di ejemplos

6 Diseño de Sistema: Lista con Acceso $O(1)$ al Final

Implementación mejorada de ListaEnlazada con referencia a cola que incluya:

(1) Diagrama de la nueva estructura

ListaEnlazada

```
├── cabeza (head) → Nodo1 → Nodo2 → ... → NodoN
└── cola (tail) → NodoN
```

Cada Nodo:

```
├── valor
└── siguiente (next)
```

(2) Código modificado con todos los métodos actualizados para mantener 'cola' consistente

```
public class ListaEnlazada<T> {
    private Nodo<T> cabeza;
    private Nodo<T> cola; // Nuevo atributo para referencia a cola
    private int tamano;

    private static class Nodo<T> {
        T valor;
        Nodo<T> siguiente;

        Nodo(T valor) {
            this.valor = valor;
            this.siguiente = null;
        }
    }
}
```

```

public ListaEnlazada() {
    cabeza = null;
    cola = null;
    tamano = 0;
}

// Insertar al inicio
public void insertarInicio(T valor) {
    Nodo<T> nuevo = new Nodo<>(valor);
    nuevo.siguiente = cabeza;
    cabeza = nuevo;
    if (tamano == 0) {
        cola = nuevo; // La cola también apunta al único nodo
    }
    tamano++;
}

// Insertar al final - ¡O(1)!
public void insertarFinal(T valor) {
    Nodo<T> nuevo = new Nodo<>(valor);
    if (tamano == 0) {
        cabeza = nuevo;
        cola = nuevo;
    } else {
        cola.siguiente = nuevo; // Mantiene la consistencia
        cola = nuevo;
    }
    tamano++;
}

// Eliminar al inicio
public T eliminarInicio() {
    if (tamano == 0) throw new RuntimeException("Lista vacía");
    T valor = cabeza.valor;
    cabeza = cabeza.siguiente;
    tamano--;
    if (tamano == 0) cola = null;
    return valor;
}

// Eliminar al final
public T eliminarFinal() {
    if (tamano == 0) throw new RuntimeException("Lista vacía");

```



```

T valor = cola.valor;
if (tamano == 1) {
    cabeza = null;
    cola = null;
} else {
    Nodo<T> actual = cabeza;
    while (actual.siguiente != cola) {
        actual = actual.siguiente;
    }
    actual.siguiente = null;
    cola = actual;
}
tamano--;
return valor;
}

```

```

public int tamano() {
    return tamano;
}

```

```

// Mostrar lista (para pruebas)
public void mostrar() {
    Nodo<T> actual = cabeza;
    while (actual != null) {
        System.out.print(actual.valor + " -> ");
        actual = actual.siguiente;
    }
    System.out.println("NULL");
}
}

```

(3) Análisis de cómo cambió la complejidad de cada operación
InsertarInicio()

Complejidad: $O(1)$

Explicación: Se crea un nuevo nodo y se coloca al inicio, sin recorrer la lista.

Sin cambios con respecto a la versión anterior.

InsertarFinal()

Complejidad: $O(1)$ (mejorada)

Explicación: Gracias a la referencia directa a la cola, se agrega el nuevo nodo al final en tiempo constante, sin necesidad de recorrer la lista.

Antes: $O(n)$ (recorriendo toda la lista para encontrar el último nodo).

Ahora: $O(1)$.

eliminarInicio()

Complejidad: $O(1)$

Explicación: Se elimina el primer nudo y se actualiza la referencia a cabeza.

Sin cambios con respecto a la versión anterior.

eliminarFinal()

Complejidad: $O(n)$

Explicación: Hay que recorrer la lista desde la cabeza para encontrar el nodo anterior a la cola, ya que la estructura es simple (no doblemente enlazada).

Sin cambios con respecto a la versión anterior.

mostrar()

Complejidad: $O(n)$

Explicación: Recorre todos los nodos para mostrarlos.

Sin cambios con respecto a la versión anterior.

(4) Pruebas que demuestren que insertarFinal() ahora es $O(1)$

```
public class ListaEnlazadaTest {  
    public static void main(String[] args) {  
        ListaEnlazada<Integer> lista = new ListaEnlazada<>();  
  
        // Insertar al final  
        lista.insertarFinal(10);  
        lista.insertarFinal(20);  
        lista.insertarFinal(30);  
        lista.mostrar(); // Esperado: 10 -> 20 -> 30 -> NULL  
  
        // Insertar al inicio (para verificar integridad)  
        lista.insertarInicio(5);  
        lista.mostrar(); // Esperado: 5 -> 10 -> 20 -> 30 -> NULL  
  
        // Eliminar inicio y final  
        lista.eliminarInicio();  
        lista.eliminarFinal();  
        lista.mostrar(); // Esperado: 10 -> 20 -> NULL  
  
        // Prueba inserciones finales en tiempo  $O(1)$   
        long start = System.nanoTime();  
        for (int i = 0; i < 1_000_000; i++)  
            lista.insertarFinal(i);  
        long end = System.nanoTime();
```

```
        System.out.println("Tiempo en insertarFinal 1,000,000 elementos: "
+ (end-start)/1e6 + " ms");
    }
}
```

Checkpoint Metacognitivo - Profundiza

¿Puedo identificar al menos una situación en mi vida académica o laboral donde una lista enlazada sería la estructura correcta?

Aún no, aunque lo tengo más claro no puedo interpretar bien los análisis de la complejidad

¿Entiendo por qué una referencia a 'cola' hace insertarFinal() $O(1)$ pero no hace eliminarFinal() $O(1)$?

insertarFinal() $O(1)$:

Como tienes un acceso directo al último nodo ("cola"), puedes agregar otro nodo al final de la lista rápidamente, sin buscar ni recorrer la lista.

eliminarFinal() $O(n)$:

Solo puedes llegar rápido al último nodo, pero para eliminarlo necesitas encontrar el penúltimo nodo. Como no hay un enlace hacia atrás, tienes que recorrer toda la lista desde el principio para localizar, por eso esta operación tarda más.

¿Qué preguntas me quedan pendientes que quiero investigar o preguntar la próxima semana?

¿Cómo hacer que eliminarFinal() sea $O(1)$?

¿Cuándo es útil mantener referencias adicionales (como a cola, oa penúltimo nodo)?